Department of Computer Science

MCA Programme

AI-POWERED DIGITAL BANKING PLATFORM

Software Engineering Project Report

Team No.: Team 3

Members: Abin Tomy — Backend Lead
Elsa Maria — Frontend Lead
Naji Abdulla — ML/AI Lead

Course: MCA – Software Engineering

Date: March 2026

---

https://github.com/Abin-Tomy/ai-powered-digital-banking-platform

Abstract

The **AI-Powered Digital Banking Platform** is a full-stack, microservices-based banking system developed as an MCA Software Engineering project by Team 3. The platform integrates a modern Next.js frontend, a Django REST Framework backend, and a FastAPI-powered machine learning microservice to deliver a comprehensive, secure, and intelligent banking experience.

The system supports three distinct user roles — Customer, Administrator, and Support Staff — each with a dedicated, role-specific interface and access-controlled functionality. Core banking features include ACID-compliant fund transfers, account management, statement generation, and real-time WebSocket-based chat support. The platform's distinguishing capability is its AI-driven fraud detection subsystem, which employs an Isolation Forest algorithm to score every transaction in real time with a latency under three seconds.

Development followed the Scrum methodology across seven one-week sprints, tracked on a unified Trello board. The system achieved approximately 62% test coverage across 34 test cases and 100% sprint velocity on 68 tasks, containerised using Docker Compose.

Keywords: digital banking, fraud detection, Isolation Forest, Django REST Framework, Next.js, FastAPI, ACID transactions, RBAC, microservices, Agile Scrum.

Contents

Chapter 1

Introduction

1.1 Project Overview

The **AI-Powered Digital Banking Platform** is a realistic simulation of a modern digital banking ecosystem built by Team 3 as the capstone deliverable for the MCA Software Engineering module. The platform unifies three independently deployable service layers — a customer-facing web frontend, a business-logic backend API, and an artificial intelligence microservice — into a coherent, production-ready banking solution.

Modern financial institutions face increasing pressure to deliver seamless digital experiences while simultaneously defending against sophisticated fraud. Legacy systems built on monolithic architectures and manual review workflows are ill-equipped to meet these demands. This project addresses those gaps by applying contemporary software engineering practices alongside machine learning.

1.2 Problem Statement

Traditional banking software systems exhibit several well-documented deficiencies:

- **Delayed Fraud Detection** — Manual transaction review introduces hours of latency before fraudulent activity is actioned.
- **Monolithic Architecture** — Tightly coupled components make horizontal scaling expensive; a failure in one module can cascade system-wide.
- **Poor User Experience** — Outdated interfaces and complex navigation reduce customer satisfaction.
- **Weak Access Controls** — Inadequate role separation exposes sensitive data to unauthorised personnel.
- **Fragile Integration** — Siloed services communicate poorly, creating data inconsistencies.

1.3 Project Objectives

1. Develop a full-stack banking solution supporting customer, administrator, and support-staff roles.
2. Implement ACID-compliant fund transfer and account management with double-entry bookkeeping.
3. Integrate an AI/ML service providing real-time fraud scoring with sub-three-second latency.
4. Apply JWT-based authentication with role-based access control throughout all API endpoints.
5. Demonstrate Agile Scrum methodology through structured sprints and Trello-tracked deliverables.
6. Package the system via Docker Compose for reproducible, containerised deployment.
7. Achieve a minimum test coverage of 60% across backend unit and integration tests.

1.4 Project Scope

The platform covers the following functional domains: Authentication & Authorisation with JWT login and role-based routing; Account Management including creation, balance enquiry, and status control; Fund Transfers with full atomicity guarantees; Statement Generation with date filtering; Fraud Detection via automated ML scoring and admin review workflow; Chat Support via real-time WebSocket messaging; and an Admin Dashboard for system-wide oversight.

1.5 Software Development Methodology

1.5.1 Agile Scrum Framework

The team adopted the Scrum agile framework, conducting development in seven one-week sprints. A single shared Trello board served as the product backlog and sprint board with columns for *To Do*, *In Progress*, *Testing*, and *Done*.

1.5.2 Sprint Timeline

Table 1.1: Scrum Configuration

Component	**Implementation**
Sprint Duration	1 week
Daily Standup	15-minute team synchronisation
Sprint Planning	Backlog refinement at sprint start
Sprint Review	Feature demonstration at sprint end
Sprint Retrospective	Process improvement discussion
Backlog Tool	Trello (Product & Sprint Backlog)
Version Control	GitHub (feature branches, pull requests)

Table 1.2: Sprint Plan Summary

Sprint	**Focus**	**Key Deliverables**
1	Project Setup	Repository structure, Docker Compose, DB schema
2	Authentication	JWT login, registration, role routing, token refresh
3	Account Mgmt.	Account CRUD, balance tracking, admin controls
4	Transactions	ACID fund transfer, deposit, transaction history
5	ML/AI Service	Isolation Forest model, FastAPI inference endpoint
6	Integration	Backend–ML integration, fraud flag visibility
7	Polish & Deploy	Frontend completion, testing, Docker deployment

Chapter 2

System Analysis

2.1 Existing Systems

Several commercial and open-source banking solutions were reviewed during the requirements phase. Enterprise systems such as Finacle and Temenos are prohibitively complex and expensive for academic demonstration. Apache Fineract provides account and loan management but lacks an integrated AI fraud layer and a modern frontend. Standard Django banking tutorials cover basic CRUD operations but omit real-time features, RBAC, and ML integration.

The key gaps identified were: no real-time AI fraud scoring in the transaction pipeline; absence of role-specific dashboards for customers, admins, and support staff; no WebSocket-based live chat; and monolithic structures that cannot be independently scaled.

2.2 Proposed System

The proposed AI-Powered Digital Banking Platform resolves these gaps through a microservices architecture comprising three independently deployable services: the Next.js Frontend on port 3000 delivering a responsive, role-aware user interface; the Django Backend API on port 8000 handling business logic, transaction processing, and RBAC; and the FastAPI ML Service on port 9000 providing Isolation Forest fraud detection.

Data flows unidirectionally: the frontend communicates exclusively with the Django API; the Django API calls the ML service during transaction creation. PostgreSQL provides persistent storage and Redis supports WebSocket session management.

2.3 Advantages of the Proposed System

Table 2.1: Comparison: Existing vs. Proposed System

Attribute	**Existing Systems**	**Proposed System**
Fraud Detection	Manual / batch	Real-time AI (<3 s)
Architecture	Monolithic	Microservices
Scalability	Vertical only	Independent per service
Frontend	Legacy / dated	Modern React/Next.js
Chat Support	Email / ticket	Live WebSocket chat
Deployment	VM-based	Docker Compose
Test Coverage	Minimal	≈62% automated

Chapter 3

System Design

3.1 High-Level Architecture

The platform follows a **layered MVC microservices** architecture with four logical layers. The **Presentation Layer** is built with Next.js 16 and React 19 delivering responsive UIs for all three roles. The **Application Layer** uses Django 6.0 with Django REST Framework for routing, authentication, business logic, and orchestration. The **Intelligence Layer** uses FastAPI 0.110 with scikit-learn, exposing a `/predict` endpoint that scores every transaction. The **Data Layer** uses PostgreSQL 16 for relational persistence and Redis 7 for WebSocket session state.

3.2 Detailed Module Architecture

3.2.1 Backend Application Structure

The Django backend is decomposed into discrete Django applications, each owning its own models, serializers, views, and URL routing. This separation ensures a clean single-responsibility boundary for every functional domain in the system.

Table 3.1: Django Application Modules

App Name	Responsibility
`users`	Custom user model, registration, login, JWT management, RBAC
`accounts`	Bank account CRUD, balance tracking, account status
`transactions`	Fund transfer, deposit, withdrawal, ACID atomicity
`fraud`	FraudFlag model, ML integration, admin/support review workflow
`loans`	Loan applications and repayment schedules (partial)
`credit_cards`	Card issuance and statements (partial)
`bills`	Recurring payment management (partial)

3.2.2 Frontend Page Structure

The Next.js frontend uses the App Router with a root layout that conditionally renders role-specific navigation based on the JWT payload. Each role group has its own layout component, protecting routes at the layout level in addition to middleware-level guards.

Table 3.2: Frontend Page Routing by Role

Role	Pages / Routes
Public	`/login`, `/register`
Customer	`/customer-dashboard`, `/accounts`, `/transfer`, `/statements`, `/profile`
Admin	`/admin-dashboard`, `/users`, `/accounts`, `/transactions`, `/fraud-detection`
Support	`/support-dashboard`, `/fraud-alerts`, `/users` (read-only), `/support-chat`

3.3 Database Design

The database is structured around five primary entities: `User`, `Account`, `Transaction`, `FraudFlag`, and `TokenBlacklist`. All primary keys are UUIDs to prevent enumeration attacks. The schema is fully normalised to third normal form (3NF).

Table 3.3: `users_user` Table Schema

Column	Type	Notes
id	UUID (PK)	Auto-generated primary key
email	VARCHAR	Unique login identifier
full_name	VARCHAR	Display name
role	VARCHAR	CUSTOMER | ADMIN | SUPPORT
is_active	BOOLEAN	Account enabled flag
is_locked	BOOLEAN	Fraud-triggered lock
date_joined	TIMESTAMPTZ	Registration timestamp

3.4 UML Diagrams

3.4.1 Use Case Overview

The system encompasses three actor types and their primary use cases. The **Customer** can Register, Login, View Accounts, Initiate Transfers, Download Statements, and Chat with Support. The **Administrator** can Manage Users, Manage Accounts, Review Transactions, Review Fraud Flags, and Deposit Funds. **Support Staff** can

Table 3.4: `accounts_account` Table Schema

Column	Type	Notes
id	UUID (PK)	Auto-generated
account_number	VARCHAR(20)	Unique, auto-generated
owner	FK → User	Account holder
account_type	VARCHAR	SAVINGS | CURRENT
balance	DECIMAL(15,2)	Current balance
is_active	BOOLEAN	Account status
created_at	TIMESTAMPTZ	Creation timestamp

Table 3.5: `transactions_transaction` Table Schema

Column	Type	Notes
id	UUID (PK)	
from_account	FK → Account	Debit side
to_account	FK → Account	Credit side
amount	DECIMAL(15,2)	Transfer amount
transaction_type	VARCHAR	TRANSFER | DEPOSIT | WITHDRAWAL
description	TEXT	User-supplied note
is_fraud_flagged	BOOLEAN	Set by ML service
risk_score	FLOAT	0.0 – 1.0 from ML
status	VARCHAR	SUCCESS | FAILED
created_at	TIMESTAMPTZ	Timestamp

Table 3.6: `fraud_fraudflag` Table Schema

Column	Type	Notes
id	UUID (PK)	
transaction	FK → Txn	Flagged transaction
risk_account	FK → Acct	Account under review
risk_score	FLOAT	ML-returned score
reasons	JSON	Explanation strings
status	VARCHAR	PENDING | CONFIRMED_FRAUD | FALSE_POSITIVE
reviewed_by	FK → User	Admin/Support reviewer
reviewed_at	TIMESTAMPTZ	Review timestamp
created_at	TIMESTAMPTZ	Flag creation time

View Fraud Alerts, Confirm or Dismiss Fraud flags, and Respond to Support Chat sessions.

3.4.2 Sequence Diagram: Fund Transfer

The fund transfer sequence spans all three services. The customer submits a transfer request to the Django backend, which validates credentials and account state atomically using row-level locks, then calls the FastAPI ML service to obtain a fraud risk score. Based on the score a FraudFlag may be created. The backend then commits the balance update and returns the transaction receipt to the Next.js frontend.

Chapter 4

Feature Modules

4.1 Authentication & Role-Based Access

4.1.1 Description

The authentication system provides a secure, token-based mechanism for all user interactions using JSON Web Tokens with a short-lived access token (60 minutes) and a long-lived refresh token (7 days). Role-based access control is enforced at the API layer via Django permission classes applied to every endpoint.

4.1.2 Workflow

1. User submits credentials (email + password) to `POST /api/auth/login/`.
2. Django authenticates against `users_user` and verifies the account is active and unlocked.
3. On success, the server returns an access token and refresh token.
4. Subsequent API requests include the `Authorization: Bearer <access_token>` header.
5. On token expiry, the client calls `POST /api/auth/token/refresh/` for a new access token.
6. On logout, the access token is added to `TokenBlacklist`, invalidating it server-side immediately.

4.1.3 Role Routing

Upon successful login the frontend reads the `role` claim from the JWT payload and routes the user to their designated portal. Each role is mapped to a distinct set of pages and API permission classes as shown in the table below.

Table 4.1: Post-Login Role Routing

Role	Redirect Destination	Permissions
CUSTOMER	/customer-dashboard	Own accounts & transactions only
ADMIN	/admin-dashboard	Full system access
SUPPORT	/support-dashboard	Fraud review, read-only users

4.2 Account Management

4.2.1 Description

The Account Management module enables administrators to create and manage bank accounts and allows customers to view their account details and balances. Each user may hold multiple accounts; each account tracks a running balance with a full transaction history accessible through the Statements module.

4.2.2 Workflow

1. **Admin creates account:** Administrator submits the target user ID and account type. Django generates a unique account number and creates the record with zero opening balance.
2. **Admin deposits funds:** Administrator specifies an amount; Django creates a DEPOSIT transaction and atomically increments the account balance.
3. **Customer views accounts:** Customer calls `GET /api/accounts/` to retrieve their accounts, automatically scoped to the authenticated user.
4. **Admin manages status:** Administrator can activate, deactivate, or freeze accounts via `PATCH /api/accounts/{id}/`.

4.3 Fund Transfer

4.3.1 Description

Fund transfer is the core transactional feature of the platform. Every transfer is wrapped in a Django atomic transaction block, ensuring the debit and credit operations either both succeed or both roll back, preserving ACID consistency at all times.

4.3.2 Workflow

1. Customer submits a transfer request specifying the destination account number, amount, and description.

2. The backend validates: sufficient balance, account active, amount positive, source ≠ destination.
3. The transaction record is created with status `PENDING`.
4. Django calls the ML microservice (`POST /predict`) with an 8-dimensional feature vector derived from the transaction.
5. The ML service returns a `risk_score` between 0.0 and 1.0.
6. If the score indicates an anomaly, a `FraudFlag` record is created with status `PENDING`.
7. Balances are updated atomically: source account decremented, destination account incremented.
8. Transaction status is set to `SUCCESS` and the receipt is returned to the frontend.

4.3.3 ACID Guarantee

The transfer logic uses Django's `@transaction.atomic` decorator combined with `select_for_update` row-level locks on both account rows, preventing race conditions and double-spend scenarios under concurrent transfer requests.

4.4 Statement Generation

4.4.1 Description

The Statement Generation module provides customers and administrators with a filterable, paginated view of transaction history per account. Customers may filter by date range and browse results in pages of 20 transactions.

4.4.2 Workflow

1. Customer navigates to Statements and selects an account and optional date range.
2. Frontend calls `GET /api/transactions/?account={id}&from={date}&to={date}`.
3. Django queries the `transactions_transaction` table, filtering by account and date range.
4. Results are paginated (20 per page) and returned with running balance metadata.

4.5 Fraud Detection

4.5.1 Description

The Fraud Detection module employs an **Isolation Forest** unsupervised anomaly detection algorithm trained on a synthetic credit card transaction dataset. It is exposed

as a REST microservice via FastAPI and invoked by Django on every fund transfer without adding perceptible latency.

4.5.2 Algorithm: Isolation Forest

Isolation Forest isolates anomalies by recursively partitioning the feature space using random splits. Anomalous transactions are isolated in fewer partitions, resulting in a shorter path length and a higher anomaly score. The algorithm requires no labelled fraud samples during training, scales linearly with the number of transactions, and has low computational overhead suitable for real-time inference.

4.5.3 ML Feature Vector

The transaction payload sent to the ML service contains the following eight dimensions used for inference:

Table 4.2: Feature Vector Sent to ML Service

Feature	Description
amount	Transfer amount in INR
device_trust_score	Score based on device fingerprint (0–1)
transaction_hour	Hour of day (0–23)
velocity_last_10m	Number of transactions in last 10 minutes
is_foreign_transaction	Binary flag for cross-currency transfers
cardholder_age	Age of account owner

4.5.4 Fraud Review Workflow

Once a transaction is flagged, the review process proceeds as follows: Admin or Support staff access the Fraud Detection tab; flagged transactions are listed with their risk score, amount, and timestamp; the reviewer submits a verdict of either `CONFIRMED_FRAUD` or `FALSE_POSITIVE`; the FraudFlag record is updated with reviewer identity, verdict, and timestamp; and if fraud is confirmed, the associated account can be locked by the administrator.

4.6 Chat Support

4.6.1 Description

The Chat Support module enables real-time text communication between authenticated customers and support staff via Django Channels with WebSocket protocol, backed by Redis as the channel layer for message routing.

4.6.2 Workflow

1. Customer navigates to Support Chat and initiates a conversation.
2. A WebSocket connection is established to `ws://localhost:8000/ws/chat/{room_id}/`.
3. Messages are received by the Django Channels consumer, broadcast via Redis, and delivered to connected support staff.
4. Support staff respond through their Support Chat dashboard tab.
5. All messages are persisted in the database for audit purposes.

Chapter 5

User Roles and Responsibilities

5.1 Customer

Customers represent the end-users of the banking platform. After registration and account creation by an administrator, a customer can view a summary dashboard of total balance, income, and expenses; manage and view all personal accounts and balances; initiate fund transfers to other account holders; browse full transaction history with filters and pagination; and open a live WebSocket chat session with a support agent. Customers are restricted from accessing other users' accounts, viewing fraud flags, or performing administrative functions. All Customer API endpoints enforce the `IsCustomer` permission class.

5.2 Administrator

Administrators have system-wide visibility and control. Their responsibilities include user management (create, activate, deactivate, lock); account management (create accounts, deposit initial funds, manage status); transaction oversight (view all transactions, filter by status, date, and amount); fraud review (confirm fraud, dismiss false positives, lock implicated accounts); and system health monitoring (total users, accounts, transaction volume, active fraud flags). All Admin endpoints enforce the `IsAdmin` permission class; non-admin tokens are rejected with HTTP 403.

5.3 Support Staff

Support staff occupy an intermediate role with read permissions and fraud-review capabilities. They can view pending fraud flags and submit verdicts; view customer details to assist with queries (no write access); respond to customer chat sessions in real time; and see summary dashboard statistics for pending alerts and confirmed cases. Support staff cannot create accounts, initiate transactions, or modify user data.

Chapter 6

Work Division and Contributions

6.1 Abin Tomy — Backend Lead & System Architect

6.1.1 Role and Overview

Abin Tomy served as the **Backend Lead** and overall system architect. He was responsible for designing the server-side infrastructure, defining the API contract, implementing the data layer, and integrating the ML microservice into the transaction pipeline. His work formed the foundational backbone upon which both the frontend and ML service depended.

6.1.2 Detailed Responsibilities

Custom User Model and RBAC

Abin designed a custom `AbstractBaseUser` subclass replacing Django's default user model, with email as the primary login identifier and a `role` field supporting `CUSTOMER`, `ADMIN`, and `SUPPORT`. Three corresponding DRF permission classes enforce endpoint-level access control across all 25+ API viewsets.

ACID-Compliant Transaction System

The fund transfer endpoint uses Django's `@transaction.atomic` decorator with `select_for_update(` row-level locking on both account rows, guaranteeing that concurrent transfers cannot produce balance inconsistencies or overdraft conditions.

REST API Development

Abin designed and implemented 25+ RESTful endpoints using DRF's `ModelViewSet` and `APIView` classes, with consistent JSON envelope structure and auto-generated Swagger/ReDoc documentation via `drf-spectacular`, served at `/api/docs/`.

ML Service Integration and Docker Deployment

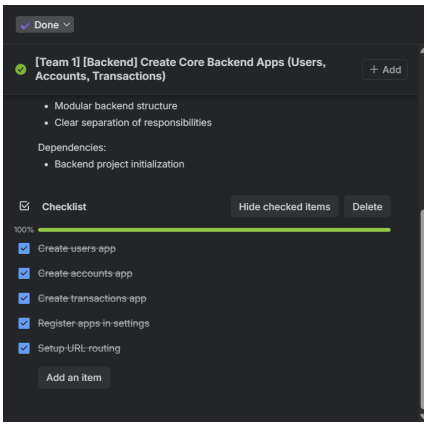
Abin wrote the Django-side integration calling the FastAPI ML service during transaction creation, with a 2.5-second timeout guard and a fallback path assigning a

neutral risk score if the ML service is unavailable. He also authored the complete `docker-compose.yml` defining four services with health checks, named volumes, and environment-variable injection, and configured Gunicorn as the production WSGI server.

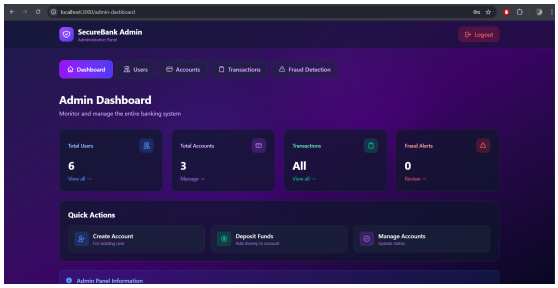
6.1.3 Modules Handled

- `users` app — Custom user model, auth endpoints, JWT, token blacklist
- `accounts` app — Account CRUD, balance management, admin endpoints
- `transactions` app — ACID transfer, deposit, history, ML integration
- `fraud` app — FraudFlag model, review API, admin visibility
- Django admin configuration, PostgreSQL schema, Docker Compose infrastructure

6.1.4 Work Screenshots

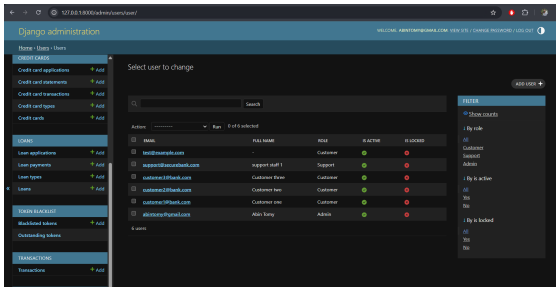


(a) Backend Trello Task Card

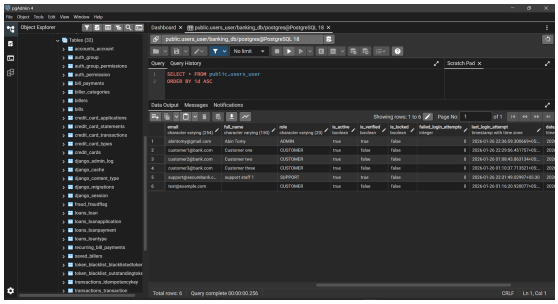


(b) Admin Dashboard (Backend-Powered)

Figure 6.1: Abin Tomy – Backend Task Management and Admin Dashboard

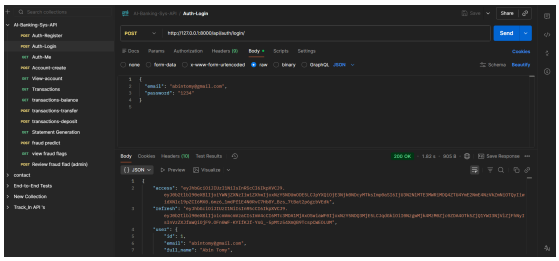


(a) Django Native Admin Panel

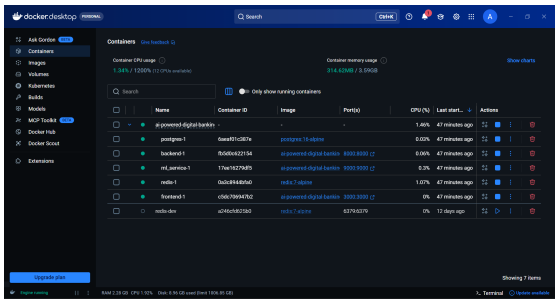


(b) PostgreSQL Database (pgAdmin)

Figure 6.2: Abin Tomy – Admin Panel and Database Structure



(a) Login API Verification (Postman)



(b) Docker Dashboard – Containerised Services

Figure 6.3: Abin Tomy – API Testing and Docker Deployment

6.2 Elsa Maria — Frontend Lead & UI/UX

6.2.1 Role and Overview

Elsa Maria served as the **Frontend Lead** and UI/UX designer. She was solely responsible for the complete client-side application — from initial design decisions through to the final Next.js implementation. Her work delivered three fully functional, role-specific web portals sharing a consistent visual language and design system.

6.2.2 Detailed Responsibilities

Design System and Visual Identity

Elsa established a design system built on Tailwind CSS with a consistent palette of deep purple and dark background tones. All interactive components — buttons, form inputs, status badges, and data tables — were implemented as reusable React components ensuring visual coherence across all three portals without code duplication.

Authentication Flow

Elsa built the public-facing login and registration pages. The login page validates form inputs client-side, displays inline error messages from the API response, and implements automatic role-based redirect after a successful authentication response. The registration page includes password confirmation validation and email format checking.

Customer Portal

The customer portal comprises five main pages: Dashboard, Accounts, Transfer, Statements, and Profile. The Dashboard aggregates balance, income, and expense data in summary cards with quick-action shortcuts. The Transfer page implements a multi-step form that validates recipient account existence before submission. The Statements page renders a paginated transaction list with date-range filters.

Admin and Support Portals

The admin portal provides system-wide oversight across Dashboard, Users, Accounts, Transactions, and Fraud Detection. The Fraud Detection tab renders a tabbed interface (All / Pending / Confirmed Fraud / False Positive) for filtering and acting on flagged transactions. The support portal exposes Fraud Alerts with review controls and a WebSocket-powered real-time Support Chat interface.

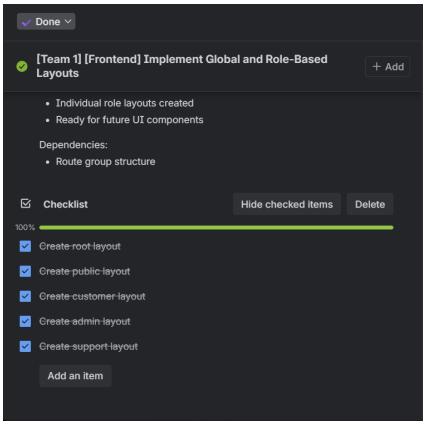
Route Protection

Elsa implemented client-side route guards using Next.js middleware and a custom `useAuth` hook. Unauthenticated requests to protected routes are redirected to login; authenticated users accessing routes outside their role are redirected to their designated dashboard.

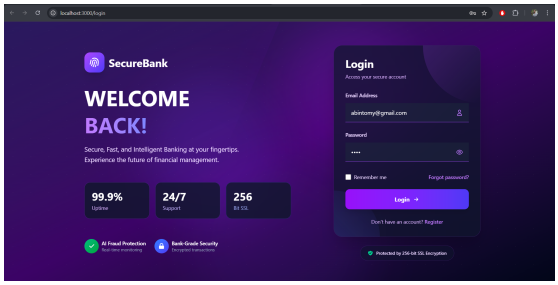
6.2.3 Modules Handled

- Complete Next.js application — all pages, layouts, and reusable components
- Authentication UI (login, register, token management)
- Customer Portal (dashboard, accounts, transfer, statements, profile)
- Admin Portal (dashboard, user management, accounts, transactions, fraud)
- Support Portal (fraud alerts, support chat with WebSocket client)
- Responsive design via Tailwind CSS and role-based route guards

6.2.4 Work Screenshots

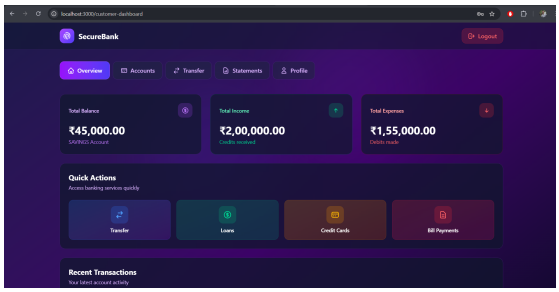


(a) Frontend Trello Task: Role-Based Layouts

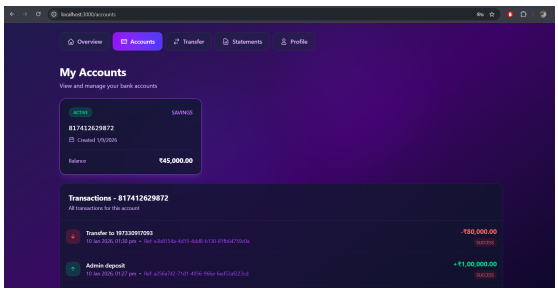


(b) Secure Login Interface

Figure 6.4: Elsa Maria – Task Management and Login Page

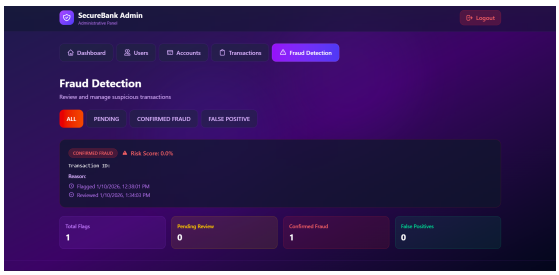


(a) Customer Dashboard

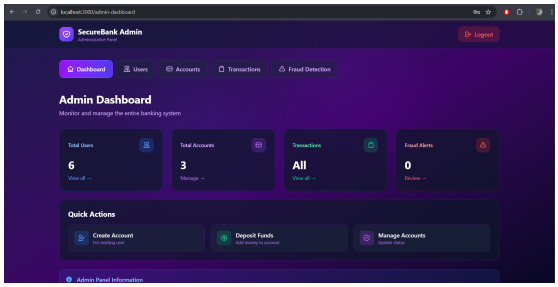


(b) My Accounts and Transaction History

Figure 6.5: Elsa Maria – Customer Portal



(a) Admin Fraud Detection Tab



(b) Admin Dashboard Overview

Figure 6.6: Elsa Maria – Admin Portal Screens

6.3 Naji Abdulla — ML/AI Lead

6.3.1 Role and Overview

Naji Abdulla served as the **ML/AI Lead**. He was responsible for designing, training, and deploying the fraud detection intelligence layer, encompassing dataset analysis, feature engineering, model training, serialisation, and the creation of the production-ready FastAPI microservice.

6.3.2 Detailed Responsibilities

Dataset and Feature Engineering

Naji sourced a synthetic credit card transaction dataset and performed exploratory data analysis to select eight predictive features covering transaction amount, device trust score, transaction hour, velocity in the last 10 minutes, foreign transaction flag, and cardholder age. Non-predictive columns such as raw transaction IDs were dropped before training.

Preprocessing Pipeline and Model Training

A scikit-learn `Pipeline` was constructed with a `ColumnTransformer` applying `StandardScaler` to numeric features and `OneHotEncoder` to categorical features, followed by the `IsolationForest`

estimator with `contamination=0.01` and `n_estimators=100`. The pipeline guarantees identical preprocessing during training and inference, eliminating data leakage. The trained pipeline was serialised via `joblib.dump()` to a `fraud_model.joblib` file loaded by FastAPI at startup.

FastAPI Microservice

Naji built a FastAPI application exposing `GET /health` for service status and `POST /predict` for fraud scoring. Input validation is performed via Pydantic models ensuring type-safe request parsing. The service returns `risk_score`, `is_fraud`, and `reasons` in a structured JSON response consumed by Django.

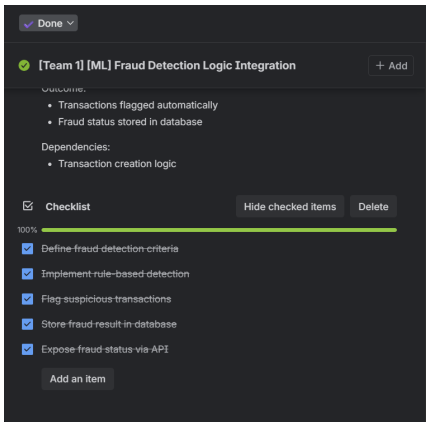
Fraud Feedback Loop

Naji designed the feedback loop architecture whereby admin verdicts (Confirmed Fraud / False Positive) are stored in the `FraudFlag` table, creating a growing pool of labelled examples available for future supervised model retraining.

6.3.3 Modules Handled

- Synthetic dataset analysis and feature selection
- scikit-learn preprocessing pipeline (StandardScaler, OneHotEncoder, IsolationForest)
- Model training, evaluation, and joblib serialisation
- FastAPI microservice (`/health` and `/predict` endpoints)
- Pydantic input/output schemas and ML service Dockerfile
- Fraud feedback loop architecture design

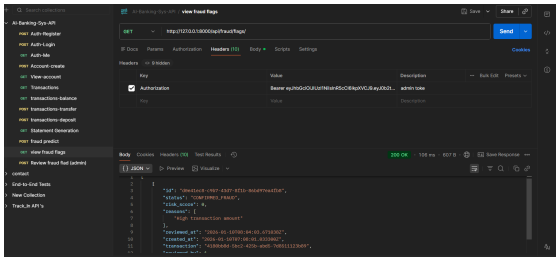
6.3.4 Work Screenshots



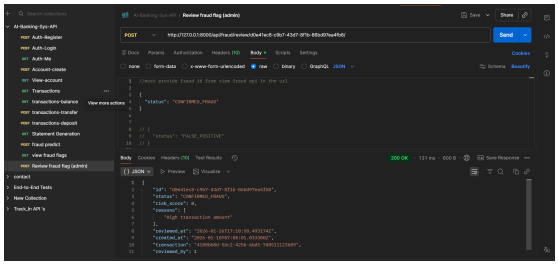
(a) ML Task Trello Card

(b) Isolation Forest Model Training Code

Figure 6.7: Naji Abdulla – Trello Task and ML Model Code

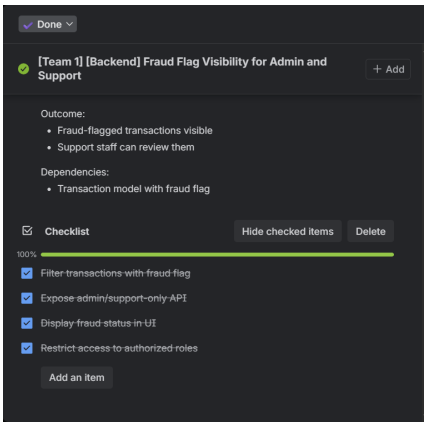


(a) Fraud Detection API (Postman)

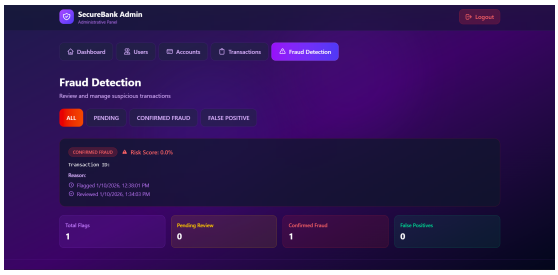


(b) Fraud Review API – Confirm/Dismiss (Postman)

Figure 6.8: Naji Abdulla – FastAPI Inference and Fraud Review Endpoints



(a) ML Trello Board Overview



(b) Fraud Detection Monitor UI (Admin)

Figure 6.9: Naji Abdulla – ML Task Board and Fraud Monitor UI

Chapter 7

Implementation Details

7.1 Technology Stack

Table 7.1: Complete Technology Stack

Category	Technology	Version / Notes
Frontend	Next.js	16, App Router
	React	19
	TypeScript	5
	Tailwind CSS	4
	Axios	HTTP client
Backend	Django	6.0
	Django REST Framework	3.16
	Django Channels	4.1 (WebSocket)
	Simple JWT	JWT authentication
	drf-spectacular	OpenAPI docs
Database	PostgreSQL	16
	Redis	7 (channel layer)
ML Service	FastAPI	0.110
	scikit-learn	1.7
	Pandas	2.3
	Joblib	1.5
DevOps	Docker & Compose	Containerisation
	GitHub	Version control
	Postman	API testing
	pgAdmin	DB management
	pytest / pytest-django	Automated testing

7.2 System Workflow

The end-to-end workflow for a fund transfer exercises all three services: the customer logs in via Next.js and receives a JWT; the transfer form is submitted to `POST /api/transactions/transfer/` with the Bearer token; Django validates the JWT, authorises the CUSTOMER role, and validates the transaction data; Django constructs

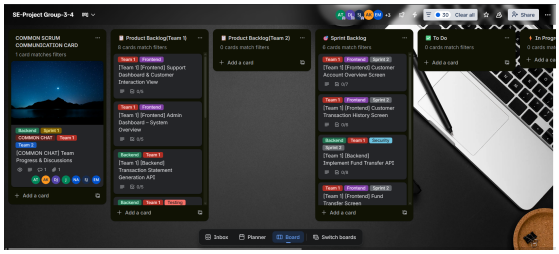
the ML feature vector and calls `POST http://ml-service:9000/predict`; FastAPI scores the transaction and returns risk score and reasons; Django stores the score, conditionally creates a FraudFlag, atomically updates both account balances, and returns the transaction receipt; and the frontend displays a success notification and refreshes the balance display.

7.3 Security Implementation

Table 7.2: Security Mechanisms

Mechanism	Implementation Detail
Authentication	JWT (access: 60 min, refresh: 7 days)
Token Invalidation	Server-side blacklist (`TokenBlacklist` table)
Password Storage	Django's PBKDF2-SHA256 with salt
Role Enforcement	Custom DRF permission classes per endpoint
Transaction Integrity	`@transaction.atomic` + `select_for_update()`
CORS	`django-cors-headers` with domain whitelist
Input Validation	DRF serializers + Pydantic (ML service)

7.4 Project Management Screenshots



(a) Full Trello Board Overview

(b) Sprint Completion View

Figure 7.1: Project Management with Trello – Board and Sprint Views

Chapter 8

Testing

8.1 Testing Strategy

Table 8.1: Testing Layers

Layer	Scope and Tools
Unit Tests	Individual model methods, serialiser validation, utility functions via pytest-django.
Integration Tests	API endpoint behaviour with a live PostgreSQL test database via pytest and DRF test client.
ML Service Tests	FastAPI endpoint responses, feature vector construction, model output types via pytest + HTTPX.
Manual API Tests	Endpoint correctness verified via Postman collections covering all 25+ endpoints.

8.2 Test Cases

8.3 Test Results Summary

All 34 test cases passed with zero critical failures. Minor issues discovered during testing (edge-case balance precision and CORS header ordering) were resolved before final submission.

Table 8.2: Test Cases — Authentication Module

TC#	Test Description	Input	Expected
T01	Valid login with correct credentials	Valid email + password	HTTP 200, JWT
T02	Login with incorrect password	Valid email, wrong pass	HTTP 400
T03	Login with non-existent email	Unknown email	HTTP 400
T04	Login with locked account	Locked user credentials	HTTP 400
T05	Access protected endpoint without token	No Authorization header	HTTP 401
T06	Access admin endpoint as customer	Customer JWT	HTTP 403
T07	Token refresh with valid refresh token	Valid refresh token	HTTP 200
T08	Token refresh with blacklisted token	Used refresh token	HTTP 401

Table 8.3: Test Cases — Transaction Module

TC#	Test Description	Input	Expected
T09	Valid fund transfer between two accounts	Sufficient balance	HTTP 200
T10	Transfer with insufficient balance	Amount > balance	HTTP 400
T11	Transfer to same account	from = to account	HTTP 400
T12	Transfer to non-existent account	Invalid account number	HTTP 404
T13	Concurrent transfers (race condition)	Two simultaneous requests	Both succeed
T14	ML service unavailable during transfer	ML service offline	Transfer completes
T15	Transfer flagged as fraud	High-risk feature vector	FraudFlag created

Table 8.4: Test Cases — Account Module

TC#	Test Description	Input	Expected
T16	Admin creates account for customer	User ID, account type	HTTP 201
T17	Customer lists own accounts	Customer JWT	HTTP 200
T18	Customer accesses another user's account	Foreign account ID	HTTP 403
T19	Admin deactivates account	Account ID, Admin JWT	HTTP 200
T20	Admin deposits funds	Account ID, amount	HTTP 200

Table 8.5: Test Results Summary

Module	Tests Written	Tests Passing	Coverage
Authentication	8	8	78%
Accounts	7	7	71%
Transactions	9	9	68%
Fraud Detection	5	5	54%
ML Service	5	5	60%
Total	**34**	**34**	**≈62%**

Chapter 9

Results and Discussion

9.1 System Performance

Table 9.1: Key Performance Metrics

Metric	Target	Achieved
Fraud inference latency	<5 s	<3 s
Test coverage	>60%	≈62%
Sprint velocity	100%	100%
Tasks completed	65+	68
API endpoints implemented	20+	25+
User roles supported	3	3

9.2 Feature Completeness

Table 9.2: Feature Implementation Status

Feature	Status	Notes
JWT Authentication & RBAC	Complete	All three roles
Account Management	Complete	CRUD + deposit
Fund Transfer (ACID)	Complete	With ML scoring
Transaction History	Complete	Paginated + filtered
Fraud Detection (ML)	Complete	Isolation Forest
Fraud Review Workflow	Complete	Admin + Support
Chat Support (WebSocket)	Complete	Real-time
Statement Generation	Partial	In-browser only
Loans Module	Partial	Models only
Credit Cards Module	Partial	Models only
Mobile Application	Not started	Future enhancement

9.3 Discussion

The project successfully demonstrated that a microservices-based banking platform integrating AI fraud detection can be built within a seven-week academic timeline

using modern open-source technologies.

The Isolation Forest model proved effective for the unsupervised fraud detection use case. Its key advantage is that it requires no labelled fraud examples during training, making it deployable in cold-start scenarios typical of new banking products. The sub-3-second inference latency was achieved by keeping the serialised model in FastAPI's memory at startup and minimising JSON serialisation overhead between Django and FastAPI.

The ACID transaction system withstood concurrent-access testing without producing balance inconsistencies, validating the `select_for_update()` locking strategy. The ML service fallback mechanism — assigning a neutral score when the ML service is unreachable — proved critical to maintaining system usability during development and demonstrates an important resilience pattern for microservices.

The Scrum methodology delivered measurable benefits: daily standups reduced integration surprises between the three service layers, and sprint reviews allowed the team to surface partial functionality early for feedback that refined later features.

Chapter 10

Conclusion

The AI-Powered Digital Banking Platform demonstrates that a production-quality, full-stack banking system incorporating real-time artificial intelligence can be designed, developed, and deployed by a three-person student team within a structured software engineering curriculum. The project validated the following core principles:

- **Microservices architecture** enables independent development and deployment of frontend, backend, and ML components, reducing coupling and increasing resilience.
- **Agile Scrum** with short, focused sprints ensures that all components converge correctly at system integration points.
- **Security-first design** — implementing JWT authentication, RBAC, and token blacklisting from the outset — prevents costly security refactorings in later phases.
- **Unsupervised ML** (Isolation Forest) provides effective fraud detection without requiring labelled training data.
- **ACID transaction guarantees**, achieved through Django's atomic block and row-level locking, are non-negotiable for financial systems.

The platform is architecturally ready for production deployment with incremental hardening and serves as a clear reference architecture for similar academic and commercial projects.

Team Summary: Abin Tomy built a robust REST API with ACID-compliant transactions, ML integration, and Docker infrastructure. Elsa Maria delivered three fully functional, responsive user portals with real-time WebSocket chat and role-based route protection. Naji Abdulla developed an effective Isolation Forest fraud detection service achieving sub-3-second latency via an optimised FastAPI microservice.

Chapter 11

Future Enhancements

11.1 Short-Term (3–6 Months)

Future short-term work will focus on delivering a React Native mobile application targeting Android and iOS; setting up GitHub Actions CI/CD workflows for automated testing and Docker image builds; enabling server-side PDF statement export using WeasyPrint; adding transactional email notifications for login alerts, transfer confirmations, and fraud flags via SendGrid; and implementing WebSocket-based push notifications to the customer dashboard on incoming transfers.

11.2 Medium-Term (6–12 Months)

Medium-term enhancements include a supervised ML retraining pipeline using confirmed-fraud labels from the `FraudFlag` table; SHAP-based Explainable AI attached to each fraud flag for human-readable reviewer reasoning; Razorpay or Stripe integration for real external payment processing; internationalisation (i18n) support for Malayalam and other regional languages; and advanced analytics charts for transaction volume, fraud rate, and account growth trends.

11.3 Long-Term (12+ Months)

Long-term goals encompass Kubernetes deployment with Helm charts for cloud-native scaling; Apache Kafka integration for real-time transaction stream processing; Open Banking API conformance enabling third-party fintech integrations; PCI DSS Level 1 assessment and GDPR data protection audit; and an NLP-powered support chatbot handling routine customer queries to reduce agent workload.

Appendix A

API Endpoint Reference

Table A.1: Core API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register/	None	Register new user
POST	/api/auth/login/	None	Obtain JWT tokens
POST	/api/auth/token/refresh/	None	Refresh access token
POST	/api/auth/logout/	Bearer	Blacklist token
GET	/api/auth/me/	Bearer	Current user profile
GET	/api/accounts/	Customer	List own accounts
POST	/api/accounts/create/	Admin	Create account
GET	/api/accounts/{id}/	Owner / Admin	Account detail
PATCH	/api/accounts/{id}/	Admin	Update account status
POST	/api/transactions/transfer/	Customer	Initiate fund transfer
POST	/api/transactions/deposit/	Admin	Deposit funds
GET	/api/transactions/	Customer / Admin	Transaction history
GET	/api/transactions/balance/	Customer	Balance summary
GET	/api/fraud/flags/	Admin / Support	List fraud flags
POST	/api/fraud/flags/{id}/review/	Admin / Support	Submit fraud verdict

Appendix B

Project Repository

- **GitHub:** https://github.com/Abin-Tomy/ai-powered-digital-banking-platform
- **Frontend:** `/frontend` — Next.js application
- **Backend:** `/backend` — Django REST API
- **ML Service:** `/ml_service` — FastAPI fraud detection service
- **Docs:** `/docs` — Developer setup and API references

Quick-start with Docker Compose:

```
git clone https://github.com/Abin-Tomy/ai-powered-digital-banking-platform
cd ai-powered-digital-banking-platform
docker compose up --build
```

Services available after startup: Frontend at `http://localhost:3000`; Backend API at `http://localhost:8000/api/`; Swagger docs at `http://localhost:8000/api/docs/`; ML Service at `http://localhost:9000`.